

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 1 Part A — Uninformed Search Algorithms

Student Name: Boniphace Benjamin Makoga

Student ID: 75051018

GitHub Repository Link: https://github.com/boniphacebenja78-lgtm/LAB_PART_A-LAB_PART_B.git

Date Submitted: 6/20/2025

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	Yes
Estimated percentage of the work influenced by AI	9%
Did you attach evidence of AI use where applicable?	yes

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
Gemini	To understand well the concept, and how to come about my assignment.		Part a and part b	
Gemini	To correct errors		Part b	

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft	yes	Attached here
Prompt history	Yes	Attached here
Screenshot of AI interaction	yes	Attached here
Exported AI conversation	yes	Attached here

AI-generated code snippet	yes	Attached here
Revised version showing student input	yes	Attached here

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	Yes
Any use of AI tools has been clearly declared in this form.	Yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	Yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	Yes
My AI usage does not exceed 25% of the entire work.	Yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	yes

Student Signature: b.b.makoga

Date: 06/20/2026

EVIDENCE,

1.

so, give me a simple explanation on all the packages am using to get familiar with before starting the assignment

To help you feel completely comfortable before diving into your assignment, here is a simple breakdown of the main Python tools and concepts you are using. You don't need to be an expert in them; you just need to understand what "job" each one handles for your drone search project.

1. `collections.deque` (The Double-Ended Queue)

- **What it is:** A smart, high-performance list.
- **Its Job:** It acts as your **Frontier** (the list of places the drone wants to explore next).
- **Why we use it:** Regular Python lists are slow when you remove things from the very front. A `deque` (pronounced "deck") is lightning-fast at both adding and removing items from either end.
 - For **BFS**, we use it as a **FIFO** queue (First-In, First-Out): we add to the back, and pull from the front.
 - For **DFS**, we use it as a **LIFO** stack (Last-In, First-Out): we add to the back, and pull from the back.

2. `abc` (Abstract Base Classes)

- **What it is:** A blueprint maker for your code.

+ Uliza Gemini

Quick ▾



Gemini is AI and can make mistakes.

so it means that zero it passes and one is a block



Exactly! You've got it perfectly.

- **0 = Pass / Open Space:** The drone can move freely through these cells.
- **1 = Block / Wall:** The drone cannot enter these cells; it has to find a way around them.

The entire maze is just a game of connecting a continuous path of **0**s from the top-left corner all the way to the bottom-right corner without running into the **1**s.

Since you have your first 25×25 map ready to go, what kind of layout do you want to design for your **Second Map** (`custom_grid_2`)? We can make it completely different—for example, a wide-open map with just a few massive block obstacles, or a tight grid with lots of parallel "rooms."



is there any structure that you write these zeros and ones, explain to me



Yes, there is a very deliberate structural system used to build these grids! When designing a grid for a pathfinding algorithm, you aren't just placing random numbers; you are creating a matrix coordinate system that the code reads like a map.

Think of the grid as a giant chessboard or a spreadsheet with rows and columns.

1. The Matrix Structure (Rows and Columns)

The grid is a **list of lists** .

- The outer brackets `[]` hold the entire map.
- Each inner list represents **one single horizontal row** from top to bottom.
- Each number inside an inner list represents **one single vertical column** from left to right.

For a 25×25 map, you have exactly 25 inner lists (rows), and each list contains exactly 25 numbers (columns).

2. The Coordinate System

The computer reads positions as `(row, column)` coordinates, starting from `0` up to `24` .

+ Uliza Gemini

Quick ▾



Gemini is AI and can make mistakes

thanks everything is ok, now how can i create my own map



That is awesome! Congratulations on getting all four search algorithms running perfectly.

Creating your own custom drone map is really fun and easy. Your maps are built using a 2D **binary grid** (a list of lists) inside Python, where:

- 0 represents an **open square** where your drone can safely fly.
- 1 represents an **obstacle/wall** that blocks the drone.

Here is a template you can paste into a new cell at the bottom of your notebook to design your own custom layout:

Python



```
# 1. Define your own grid (0 = empty, 1 = obstacle)
# This is a custom 6x6 room layout
my_custom_grid = [
    [0, 0, 0, 0, 1, 0],
    [0, 1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 0],
    [0, 1, 0, 0, 0, 0],
    [0, 0, 0, 1, 1, 0]
```



Uliza Gemini

Quick



```
-----TypeError Traceback (most recent call
last)/tmp/ipykernel_7219/1337680568.py in <cell line: 0>() 6 7
results = [----> 8 bfs.search(problem), 9 dfs.search(proble
```

This traceback points directly to a bug inside your `GridProblem.actions` method (specifically around line 49).

The error `TypeError: cannot unpack non-iterable method object` means that the variable `state` being passed into `.actions()` is actually the *method* `problem.initial_state` instead of the *evaluated coordinate tuple* `(0, 0)`.

This happens if you forgot the parentheses `()` when calling `problem.initial_state` somewhere in your search setup!

Step 1: Fix your search initialization cells

Check your BFS and DFS cells. Look at the very first step inside the `search` method:

Python



```
# ✖ INCORRECT (causes the error because it passes the method itself)
```

+ Uliza Gemini

Quick



Gemini is AI and can make mistakes.

```
----- UnboundLocalError Traceback (most recent call  
last)/tmp/ipykernel_7219/1337680568.py in <cell line: 0>() 6 7  
results = [----> 8 bfs.search(problem), 10
```

Ah, this error means inside your `BreadthFirstSearch` class, Python is trying to increment `nodes_expanded += 1`, but it can't find where `nodes_expanded` it was originally set to `0`.

This usually happens if the variable setup lines got accidentally left out or misplaced when pasting the code into your BFS cell.

How to fix it:

Go back up to the cell where you wrote your `BreadthFirstSearch` class and replace its entire contents with this clean version. Make sure that the `nodes_expanded = 0` line is sitting right at the top of the `search` method: